Generating Asset Returns with Quant GANs

Machine Learning in Finance Workshop 2020
@ Columbia University, New York

Magnus Wiese
August 7, 2020

TU Kaiserslautern

Disclaimer

Opinions and estimates constitute our judgment as of the date of this Material, are for informational purposes only and are subject to change without notice. It is not a research report and is not intended as such. Past performance is not indicative of future results. This Material is not the product of J.P. Morgan' s Research Department and therefore, has not been prepared in accordance with legal requirements to promote the independence of research, including but not limited to, the prohibition on the dealing ahead of the dissemination of investment research. This Material is not intended as research, a recommendation, advice, offer or solicitation for the purchase or sale of any financial product or service, or to be used in any way for evaluating the merits of participating in any transaction. Please consult your own advisors regarding legal, tax, accounting or any other aspects including suitability implications for your particular circumstances. J.P. Morgan disclaims any responsibility or liability whatsoever for the quality, accuracy or completeness of the information herein, and for any reliance on, or use of this material in any way. Important disclosures at: `www.jpmorgan.com/disclosures`.

Introduction

A review of the empirical properties of assets returns

The model: Quant GANs

Numerical results

Concluding remarks

Applying RL to financial datasets

Financial data on a daily scale is scarce and non-stationary. To apply deep reinforcement learning (RL) to finance training environments, for instance through market generators, need to be created as otherwise the RL agent may try to exploit spurious characteristics in the data instead of learning the underlying properties governing the data.

Potential RL applications include, but are not limited to:

- Hedging of derivatives. Buehler et al. [2019]
- Portfolio optimization.Filos [2019]
- Risk management

Synthetic market generators: related work

Recently, the topic of data-driven market simulation has attracted the attention from industry and academia:

- Fu et al. [2019], Koshiyama et al. [2019], Li et al. [2018], Wiese et al. [2019] use classic GANs to generate macroeconomic time series, stock returns, limit-order book data, and equity option markets.
- SigCWGANs Ni et al. [2020] learn the joint distribution by leveraging signatures and calibrate a deterministic discriminator before training.
- Bühler et al. [2020] Paired variational autoencoders (VAEs) with log-signatures have also been explored to generate stock returns.
- COT-GANs Xu et al. [2020] use causal optimal transport and Sinkhorn divergences.

Quant GANs

With Quant GANs we aim to approximate a synthetic market generator for asset returns in an unsupervised fashion by leveraging a neural network-based discriminator as in the classic GAN setup.

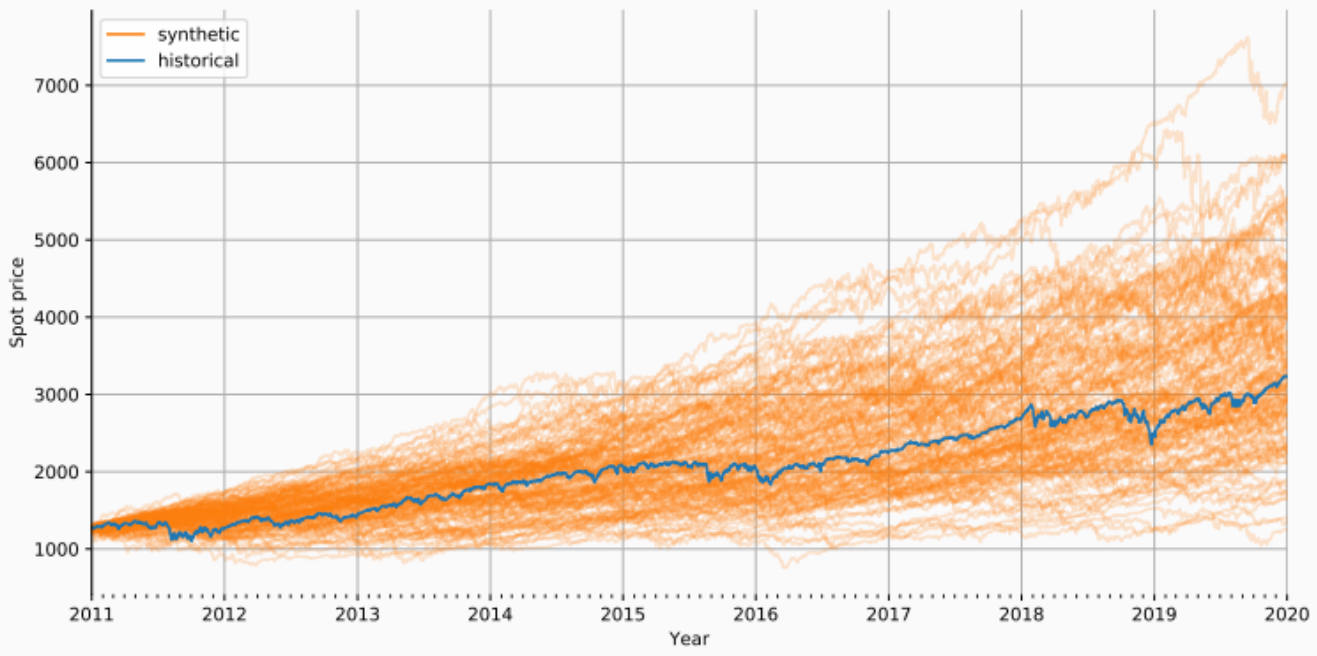

Figure 1: S&P 500 (blue) and 128 synthetic Quant GAN paths (orange).

Introduction

A review of the empirical properties of assets returns

The model: Quant GANs

Numerical results

Concluding remarks

Validating the Quant GANs performance via stylized facts

Cont [2001] identifies and summarises some of the most consistent empirical findings characterising asset returns which he coins as *stylized facts*.

Stylized facts, thus, are characteristic features we would like our synthetic market generator to have.

In what follows we illustrate a subset of the most prominent stylized facts by using the historical log-returns $(x_t)_{t=1}^{T}$ of the S&P 500.

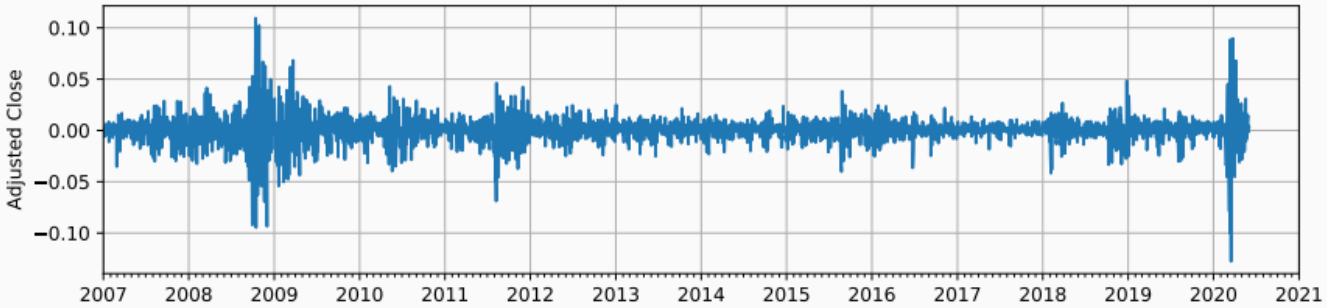

Figure 2: Adjusted close price of the S&P 500 from 3 Jan 2007 - 1 June 2020.

Stylized facts: kurtosis

Positive excess kurtosis: empirical distribution is *peaker* than the Gaussian, i.e.

$$\hat{\kappa}_h := \frac{1}{T}\sum_{t=1}^{T}\left(\frac{x_t - \bar{x}}{\hat{\sigma}(x)}\right)^4 - 3$$

where $\bar{x}$ is the average and $\hat{\sigma}(x)$ the standard deviation of $(x_t)_{t=1}^{T}$, is greater zero.

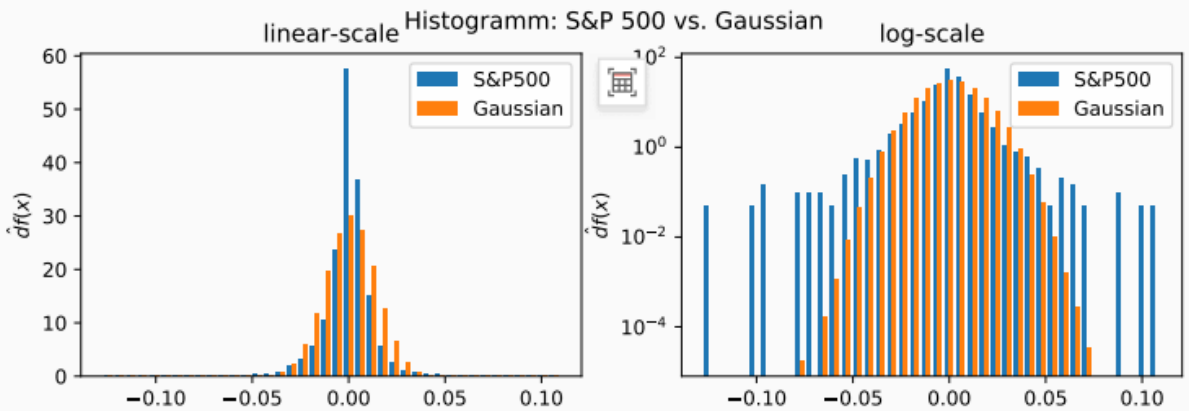

Figure 3: Unconditional distribution of the S&P 500 on the linear- (left) and log-scale. $\hat{\kappa}_h = 13.15$.

Stylized facts: fat tails

Fat-tails: tails resemble power law decay, denoting the empirical CDF by $\hat{F}_h : \mathbb{R} \to [0,1]$, we observe

$$\ln(1 - \hat{F}_h(x)) \approx -\alpha \ln(x) \quad \text{for } x \gg 0,$$

and

$$\ln \hat{F}_h(x) \approx \alpha \ln(x) \quad \text{for } x \ll 0.$$

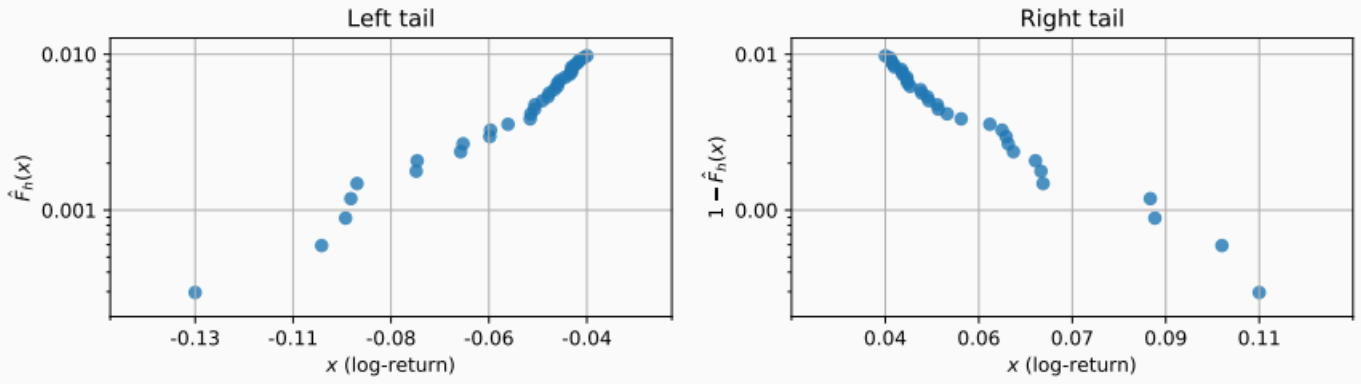

Figure 4: Left and right tail of the S&P 500. The x-axis resembles the magnitude of the log-return, and the y-axis the empirical CDF, both on the log-scale.

Stylized facts: serial autocorrelation and volatility clusters

Define the empirical auto-covariance function of $(x_t)_{t=1}^T$ as

$$\tilde{\rho}_h(k) = \frac{1}{T-k} \sum_{t=1}^{T-k} (x_t - \bar{x})(x_{t+k} - \bar{x}),$$

where $\bar{x}$ denotes the mean of $(x_t)_{t=1}^T$.[1]

No serial correlations: linear autocorrelations $\tilde{\rho}_h^x(k)/\tilde{\rho}_h^x(0)$, $k = 1, \ldots, T$ are often insignificant.

Presence of volatility clusters: phases of low and high absolute log-returns tend to cluster, i.e. $\hat{\rho}_h^{|x|}(k)/\hat{\rho}_h^{|x|}(0)$ tends to be positive and decays *slowly*.

---

[1] In our notation we use h in the subscript to indicate that we are computing the estimator with respect to historical log-returns.

Stylized facts: leverage effects

Leverage effects: anti-correlation between log-returns and volatility.

$$\hat{l}_h(k) = \frac{1}{T-k} \sum_{t=1}^{T-k} \frac{(|x_t| - \overline{|x|})(x_{t+k} - \bar{x})}{\hat{\sigma}(|x|)\hat{\sigma}(x)} < 0$$

for small h.

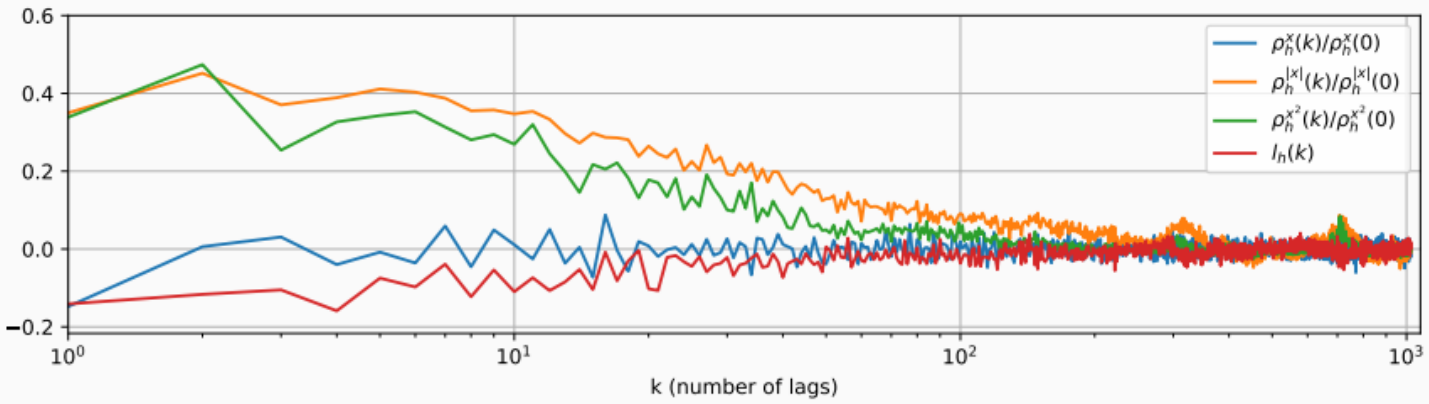

Figure 5: Autocorrelation functions of absolute, squared and serial log-returns and the leverage effect function of the S&P 500.

S&P 500: volatility clusters

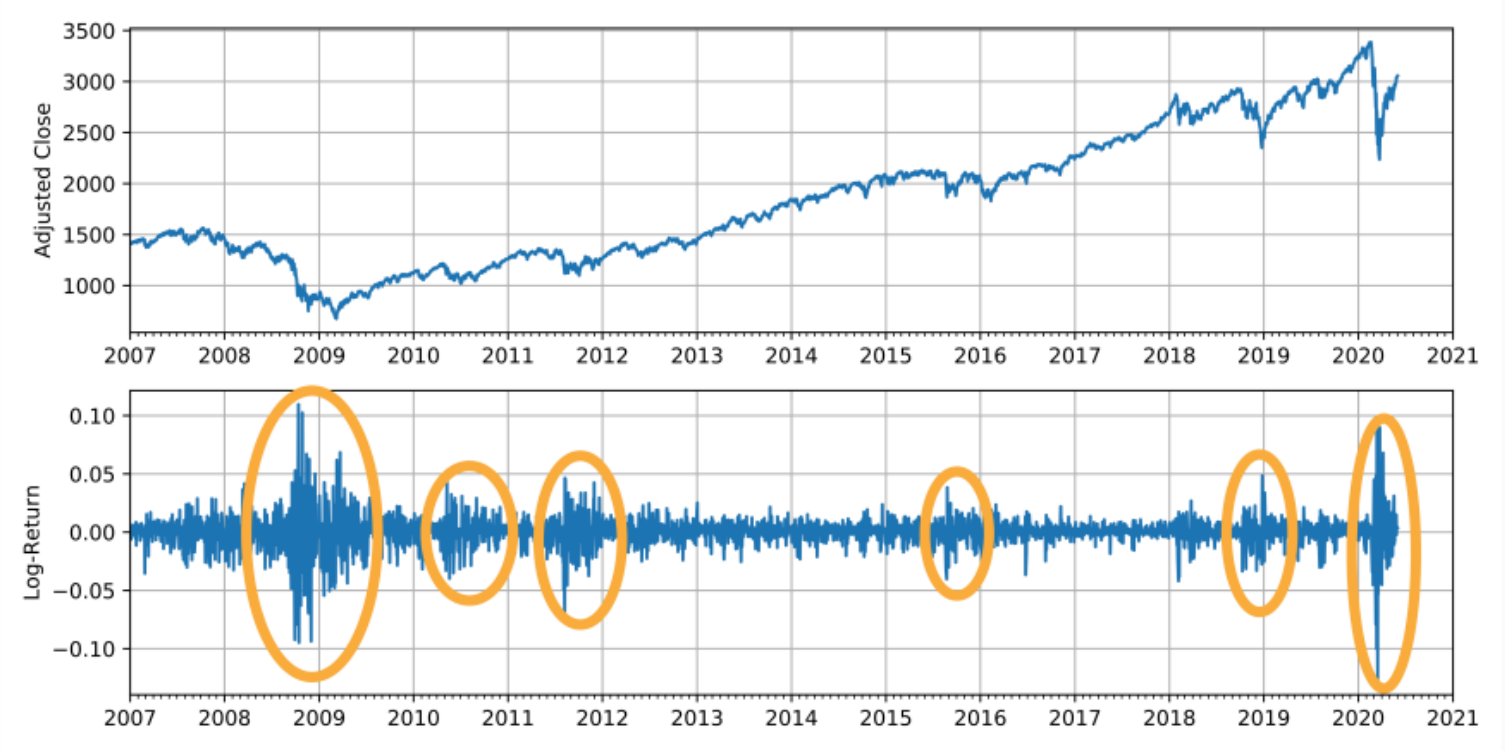

Figure 6: Spot price and log-returns of S&P 500. The orange circles highlight volatility clusters. Note that these clusters generally occur after a larger drawdown, justifying the leverage effect.

Introduction

A review of the empirical properties of assets returns

The model: Quant GANs

Numerical results

Concluding remarks

Problem formulation

Consider an unknown distribution ν and a sequence of (multivariate) log-returns $(x_t)_{t=1}^T$ in $\mathcal{X} \subseteq \mathbb{R}^d$ sampled from ν. Furthermore, assume that ν is supported on $\mathcal{Z} \subseteq \mathbb{R}^\ell$ for $d \gg \ell$.

In generative modelling the aim is to construct a generator function $G : \Theta \times \mathcal{Z} \to \mathcal{X}$ such that we can sample from an easy-to-sample distribution $\mu^{\mathcal{Z}}$ and $G^\theta_\# \mu^{\mathcal{Z}}$, $\theta \in \Theta$, and ν are *close* with respect to some metric. The challenge is to find a suitable cost function such that the parameter $\theta \in \Theta$ is learned and the distance between the two distributions is minimized.

Generative adversarial networks (GANs)

In GANs a neural network called the discriminator is utilized as this cost function. The discriminator $D : \mathcal{X} \to [0, 1]$ has the objective to discriminate whether a sample $x \in \mathcal{X}$ is drawn from the unknown data distribution ν or the generated distribution $G^{\theta}_{\#}\mu^{\mathcal{Z}}$, whereas the generator $G : \mathcal{Z} \to \mathcal{X}$ has the objective to *fool* the discriminator.

The GAN' s objective is formulated as a two-player min–max game

$$\inf_{G:\mathbb{R}^{\ell}\to\mathbb{R}^{d}} \sup_{D:\mathbb{R}^{d}\to[0,1]} \mathbb{E}_{\nu}[\ln(D \circ X)] + \mathbb{E}_{\mu^{\mathcal{Z}}}[\ln(1 - D \circ G \circ Z)] \,.$$

Note the discriminator' s objective is to minimize the binary cross-entropy.

The model: Quant GAN (1)

The Quant GAN is inspired from the recent success of generative adversarial networks (GANs) Goodfellow et al. [2014] in the image domain (see e.g. Brock et al. [2018]).

To model temporal dynamics and capture longer-ranging dependencies, the functions D and G are constructed through *temporal convolutional networks (TCNs)*. TCNs are convolutional neural networks that utilize dilated causal convolutions to capture longer-ranging dependencies.

The model: Quant GAN (2)

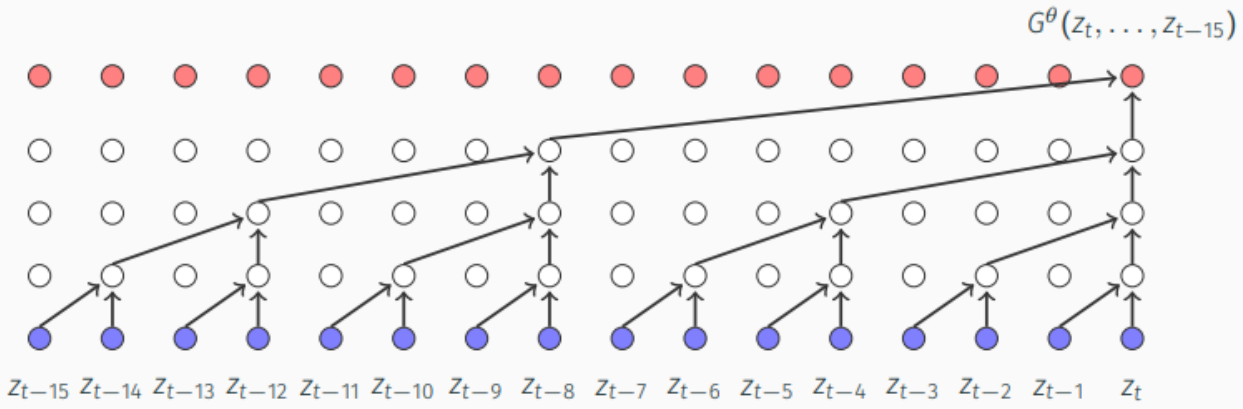

Figure 7: Depiction of a Quant GAN generator with 4 hidden layers and a dilation factor of 2. The generator's receptive field size is 16. (For details refer to [13] or to the code `github.com/magnumw/QuantGANs`.)

Model advantages and disadvantages

Advantages:
When compared to RNNs:

- No backpropagation through time required.
- Stable and parallelizable generation.
- More stable gradients due to the use of convolutions.

Disadvantages:

- Unconditional generator –no initial conditions can be inferred.
- Stochastic discriminator and no criterion when to stop training.[2]
- Not parsimonious.
- Construction of the generator causes synthetic log-returns to be independent after a finite number of lags.

---

[2] See for example Ni et al. [2020] for a deterministic discriminator which is approximated before training and also Bühler et al. [2020] which pair log-signatures with VAEs.

Introduction

A review of the empirical properties of assets returns

The model: Quant GANs

Numerical results

Concluding remarks

Test metrics: distributional

To assess the convergence and observe if the Quant GAN learns stylized facts, we compute a number of test metrics of the unconditional distribution during training.

Denote by $\hat{df}_\cdot, \hat{\kappa}_\cdot, \hat{s}_\cdot$ for $\cdot \in \{h, G\}$ the unconditional distribution, kurtosis and skew of historical and generated data respectively. We track the following test metrics:

Density metric:

$$|\hat{df}_h - \hat{df}_G|_1$$

Absolute difference of kurtosis:

$$|\hat{\kappa}_h - \hat{\kappa}_G|$$

Absolute difference of skew:

$$|\hat{s}_h - \hat{s}_G|$$

Test metrics: dependence

Furthermore, we compute different test metrics to assess the dependence structure of synthetic data.

Serial ACF metric:

$$\frac{1}{64}\sum_{h=1}^{64}\left|\frac{\tilde{\rho}_h(h)}{\tilde{\rho}_h(0)}-\frac{\tilde{\rho}_G(h)}{\tilde{\rho}_G(0)}\right|$$

Absolute difference of the ACF of absolute log-returns:

$$\frac{1}{64}\sum_{h=1}^{64}\left|\frac{\hat{\rho}_h^{|x|}(h)}{\hat{\rho}_h^{|x|}(0)}-\frac{\hat{\rho}_G^{|x|}(h)}{\hat{\rho}_G^{|x|}(0)}\right|$$

Absolute difference of the leverage effect function:

$$\frac{1}{16}\sum_{h=1}^{16}|\hat{l}_h(h)-\hat{l}_G(h)|$$

Training settings

Hyperparameters	
Batch size	512
Learning rates	1e-4 / 3e-4 (G / D)
Optimizer	Adam
Latent dimension	3
Lookback	127 / 127 (G / D)
Total number of parameters	85424 / 83824 (G / D)
Hardware	
GPU	RTX 2070
Required memory	2663MiB
Total training time	1:49 minute
Dataset	
Asset	S&P 500 closing price
Time horizon	3 Jan 2011 - 31 Dec 2019

Can Quant GANs learn stylized facts?

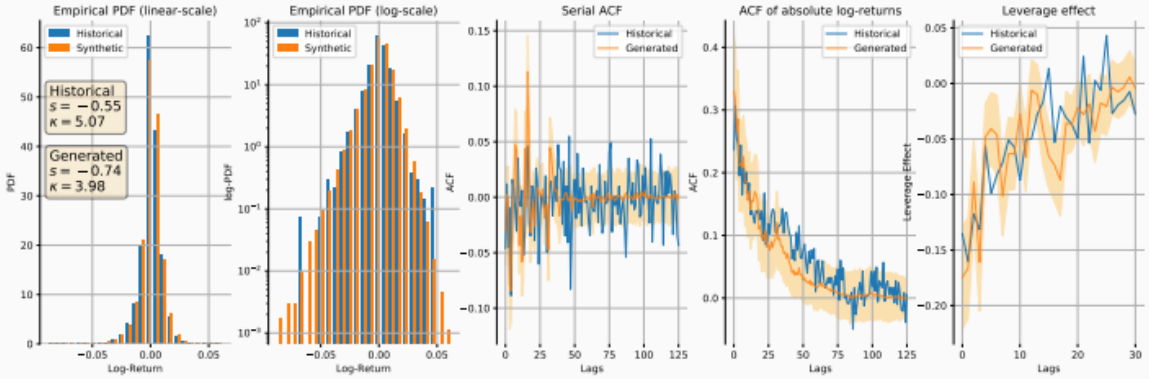

Figure 8: From left to right: comparison of the empirical PDFs on the linear- and log-scale, serial ACF, ACF of absolute log-returns and the leverage effect function. The colors blue and orange indicate the estimators computed from historical and synthetic data respectively.

Synthetic paths

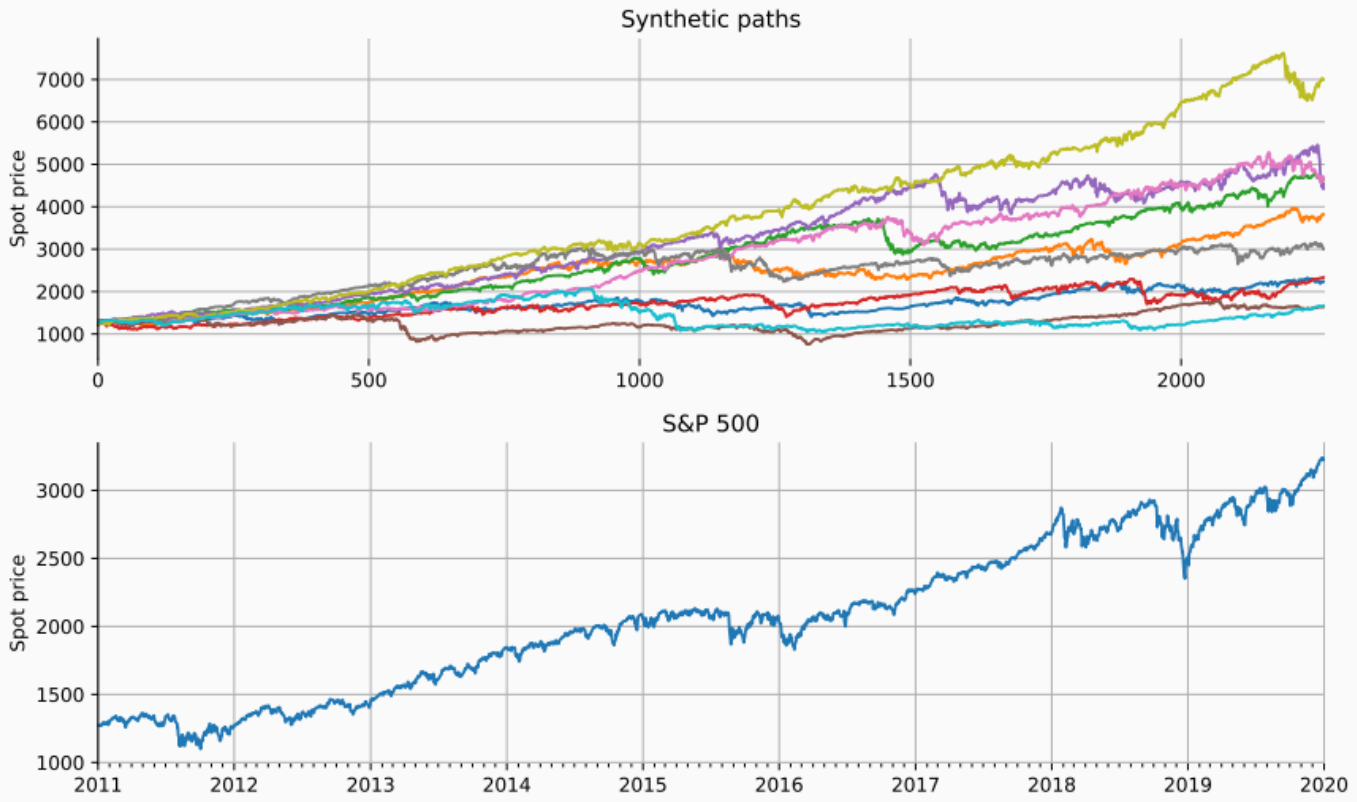

Figure 9: Ten synthetic paths (top) and the historical S&P 500 path (bottom).

How well does the model converge?

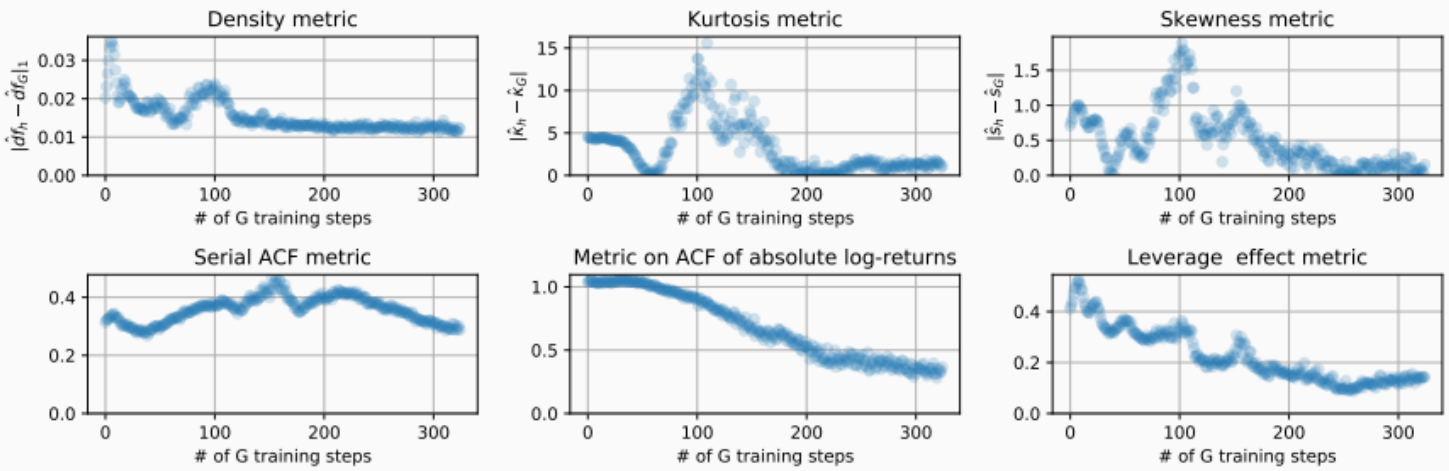

Figure 10: Development of the test metrics during training. Observe that most of the test metrics, except the serial ACF metric, follow a downward trend. This confirms that the discriminator of the Quant GAN is able to detect stylized facts.

Introduction

A review of the empirical properties of assets returns

The model: Quant GANs

Numerical results

Concluding remarks

Concluding remarks

- Numerical results demonstrate that Quant GANs can learn stylized facts in an unsupervised fashion.
- However, the unconditional structure of the model and the independence assumption (see [Wiese et al. [2020], Remark 5.3]) may limit the model's utility for further applications.
- Although Quant GANs could be trained with larger receptive fields, in practice optimisation becomes unstable, leaving future research.
- Furthermore, more elaborate tests need to be developed to test the proximity of the generated distribution to the historical, see e.g. Bühler et al. [2020], Chevyrev and Oberhauser [2018].

Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale gan training for high fidelity natural image synthesis. In *International Conference on Learning Representations*, 2018.

Hans Buehler, Lukas Gonon, Josef Teichmann, and Ben Wood. Deep hedging. *Quantitative Finance*, pages 1–21, 2019.

Hans Bühler, Blanka Horvath, Terry Lyons, Ignacio Perez Arribas, and Ben Wood. A data-driven market simulator for small data environments. Technical report, arXiv.org, 2020. arXiv:2009.08454.

Ilya Chevyrev and Harald Oberhauser. Signature moments to characterize laws of stochastic processes. *arXiv preprint arXiv:1810.10971*, 2018.

Rama Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 2001.

Alexandros Filos. Reinforcement learning for portfolio management. *arXiv preprint arXiv:1909.09571*, 2019.

Rao Fu, Jie Chen, Shutian Zeng, Yiping Zhuang, and Agus Sudjianto. Time Series Simulation by Conditional Generative Adversarial Net. *arXiv preprint arXiv:1904.11419*, 2019.

Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative Adversarial Nets. *Advances in Neural Information Processing Systems*, 27:2672–2680, 2014.

Adriano Koshiyama, Nick Firoozye, and Philip Treleaven. Generative adversarial networks for financial trading strategies: fine-tuning and combination. *arXiv preprint arXiv:1901.01751*, 2019.

Jiaqi Li, Xiaotao Wang, Yifan Lin, Ankur Sinha, and Michael P Wellman. Generating realistic stock market order streams. In *Proceedings of the International Joint Conference on Artificial Intelligence*, 2018.

Hao Ni, Lukasz Szpruch, Magnus Wiese, Shujian Liao, and Baoren Xiao. Conditional Sig-Wasserstein GANs for Time Series Generation. *arXiv preprint arXiv:2006.05421*, 2020.

Magnus Wiese, Lei Bai, Ben Wood, and Hans Buehler. Deep hedging: Learning to simulate equity option markets. *SSRN Electronic Journal*, 2019.

Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant GANs: Deep Generation of Financial Time Series. *Quantitative Finance*, pages 1–22, 2020.

Leo Xu, K. Wenliang, Mark Munn, and Beatrice Acciaio. Cot-gan: Generating sequential data via causal optimal transport. *arXiv preprint arXiv:2006.08539*, 2020.